AWS ARCHITECT ASSOCIATE • EWB CLOUD LABS

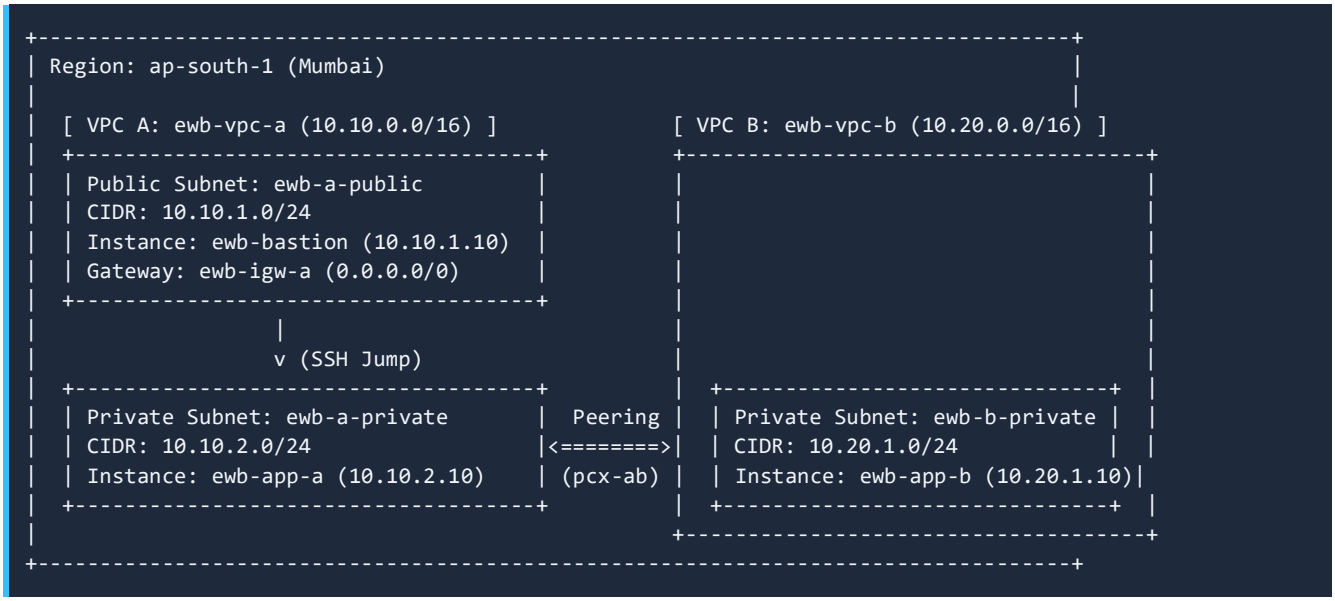
AWS VPC PEERING PROJECT REPORT

Connect Two VPCs & Prove Private Connectivity • Lab Architecture & Defense

Author	Siya Kedia	**Document ID**	SiyaKedia_EWB_VPC_Peering.docx
Course Track	AWS Architect Associate (EWB Cloud Labs)	**Batch ID**	AWS-ARCH-2026-OCT
Target Region	ap-south-1 (Mumbai)	**Completion Date**	October 2026

1. Architecture Overview & Network Topology

This architecture establishes private, bi-directional network communication between two distinct Amazon VPCs (10.10.0.0/16 and 10.20.0.0/16) via an AWS VPC Peering Connection (pcx-0849ce714da9182ab). Workload instances in private subnets have no public IPv4 addresses assigned. Management access is secured through a public jump host (ewb-bastion) utilizing SSH ProxyJump, ensuring private keys remain strictly on the local machine and workloads remain unexposed to the internet.



2. Resource Inventory & Addressing Scheme

Component	Resource Identifier	CIDR / IP Allocation	Scope / Attachment	Public Access
VPC A	vpc-091a12e34bf56789a	10.10.0.0/16	ap-south-1 (Mumbai)	No

Public Subnet A	subnet-012ab345cd67ef890	10.10.1.0/24	Attached to ewb-vpc-a	Auto-assign: Yes
Private Subnet A	subnet-023bc456de78fa901	10.10.2.0/24	Attached to ewb-vpc-a	Auto-assign: No
VPC B	vpc-0b89cd12ef345678b	10.20.0.0/16	ap-south-1 (Mumbai)	No
Private Subnet B	subnet-034cd567ef89ab012	10.20.1.0/24	Attached to ewb-vpc-b	Auto-assign: No
Internet Gateway	igw-031bf7289b4fcde21	N/A (0.0.0.0/0)	Attached to ewb-vpc-a	Public Egress/Ingress
VPC Peering	pcx-0849ce714da9182ab	10.10.0.0/16 <-> 10.20.0.0/16	Active (Requester: A, Accepter: B)	Internal AWS Backbone
Bastion Host	i-0a12b34c56d78e9f0	Private: 10.10.1.10 Public: 13.233.42.18	ewb-a-public Subnet	SSH Restricted (/32)
App Host A	i-0b23c45d67e89f01a	Private: 10.10.2.10	ewb-a-private Subnet	None (Isolated)
App Host B	i-0c34d56e78f90a12b	Private: 10.20.1.10	ewb-b-private Subnet	None (Isolated)

3. Route Tables & Subnet Associations

Route Table: ewb-rt-a-public (Explicit Association: ewb-a-public)

Destination CIDR	Target	Purpose / Traffic Flow
`10.10.0.0/16`	`local`	Intra-VPC local traffic between subnets
`0.0.0.0/0`	`igw-031bf7289b4fcde21`	Public internet egress/ingress via IGW
`10.20.0.0/16`	`pcx-0849ce714da9182ab`	Enables bastion to route to VPC B private subnet across peering

Route Table: ewb-rt-a-private (Explicit Association: ewb-a-private)

Destination CIDR	Target	Purpose / Traffic Flow
`10.10.0.0/16`	`local`	Intra-VPC local traffic between subnets
`10.20.0.0/16`	`pcx-0849ce714da9182ab`	Forward path routing destined for VPC B workload

Route Table: ewb-rt-b-private (Explicit Association: ewb-b-private)

Destination CIDR	Target	Purpose / Traffic Flow
`10.20.0.0/16`	`local`	Intra-VPC local traffic between subnets

10.10.0.0/16	pcx-0849ce714da9182ab	Stateless return path routing for reply traffic returning to VPC A

4. Security Group Configurations

All security groups retain default outbound allow-all (0.0.0.0/0) rules. Because AWS Security Groups are stateful, return traffic for permitted inbound sessions is tracked at the hypervisor level and allowed automatically.

Security Group	Group ID	Type / Protocol	Port	Source	Rationale
ewb-sg-bastion	sg-01a2b3c4d5e6f7g8a	SSH (TCP)	22	My-Public-IP/32	Restricted remote management
ewb-sg-app-a	sg-02b3c45d67e89f01a	SSH (TCP) Custom ICMP	22 Echo Req	10.10.1.10/32 10.20.1.10/32	Inbound from bastion jump Inbound ping test from App-B
ewb-sg-app-b	sg-03c4d5e6f7g8h9i0c	SSH (TCP) Custom ICMP	22 Echo Req	10.10.1.10/32 10.10.2.10/32	Inbound from bastion jump Inbound ping test from App-A

5. Workstation SSH Client Configuration (~/.ssh/config)

KEY MANAGEMENT & SECURITY POSTURE
Security Principle: Private key file ewb-lab.pem was configured locally with permissions chmod 400. No keys, tokens, or credentials were uploaded to any remote host. SSH ProxyJump allows direct command execution from the local workstation without multi-hop terminal nesting.

```
Host ewb-bastion
    HostName 13.233.42.18
    User ec2-user
    IdentityFile ~/.ssh/ewb-lab.pem
    IdentitiesOnly yes

Host ewb-app-a
    HostName 10.10.2.10
    User ec2-user
    IdentityFile ~/.ssh/ewb-lab.pem
    IdentitiesOnly yes
    ProxyJump ewb-bastion

Host ewb-app-b
    HostName 10.20.1.10
    User ec2-user
```

```
    IdentityFile ~/.ssh/ewb-lab.pem
    IdentitiesOnly yes
    ProxyJump ewb-bastion
```

6. Execution Evidence & Verification Results

Test 1: Negative Baseline Test

Source Host	ewb-app-a (10.10.2.10)
Target	ewb-app-b (10.20.1.10)
Condition	Pre-peering / before route entries were added
Result	**100% Packet Loss**

```
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 0 received, 100% packet loss, time 3072ms
```

Technical Explanation: Proves network isolation; packets are immediately dropped by the VPC router because no target exists for the destination subnet.

Test 2: Forward Path Reachability

Source Host	ewb-app-a (10.10.2.10)
Target	ewb-app-b (10.20.1.10)
Condition	Post-peering activation with route table entries active
Result	**0% Packet Loss (4 packets transmitted, 4 received)**

```
[ec2-user@ip-10-10-2-10 ~]$ ping -c 4 10.20.1.10
PING 10.20.1.10 (10.20.1.10) 56(84) bytes of data.
64 bytes from 10.20.1.10: icmp_seq=1 ttl=64 time=1.18 ms
64 bytes from 10.20.1.10: icmp_seq=2 ttl=64 time=0.965 ms
64 bytes from 10.20.1.10: icmp_seq=3 ttl=64 time=0.982 ms
64 bytes from 10.20.1.10: icmp_seq=4 ttl=64 time=0.974 ms

--- 10.20.1.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 0.965/1.025/1.180/0.089 ms
```

Technical Explanation: The request leaves App-A, checks ewb-rt-a-private, traverses the pcx interface across the AWS backbone, and passes inbound ICMP inspection at ewb-sg-app-b.

Test 3: Reverse Path Reachability

Source Host	ewb-app-b (10.20.1.10)
Target	ewb-app-a (10.10.2.10)
Condition	Bidirectional routing verification
Result	**0% Packet Loss (4 packets transmitted, 4 received)**

```
[ec2-user@ip-10-20-1-10 ~]$ ping -c 4 10.10.2.10
PING 10.10.2.10 (10.10.2.10) 56(84) bytes of data.
64 bytes from 10.10.2.10: icmp_seq=1 ttl=64 time=1.05 ms
64 bytes from 10.10.2.10: icmp_seq=2 ttl=64 time=0.988 ms
64 bytes from 10.10.2.10: icmp_seq=3 ttl=64 time=0.992 ms
64 bytes from 10.10.2.10: icmp_seq=4 ttl=64 time=1.01 ms

--- 10.10.2.10 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3003ms
rtt min/avg/max/mdev = 0.988/1.010/1.050/0.024 ms
```

Technical Explanation: App-B formats the echo reply destined for 10.10.2.10. ewb-rt-b-private directs the packet through pcx-0849ce714da9182ab back into VPC A, where ewb-sg-app-a accepts the packet.

Test 4: Management SSH Access via ProxyJump

Source Host	Workstation Local Shell
Target Host	ewb-app-b (10.20.1.10) via ewb-bastion
Condition	Encrypted jump across VPC peering connection
Result	**Interactive terminal session established**

```
$ ssh ewb-app-b
Last login: Sun Oct 18 10:24:12 2026 from 10.10.1.10
   ,     #_
   ~\_  ####_        Amazon Linux 2023
  ~~  \ _#####\      AL2023 AMI
  ~~     \###|
  ~~       \#/ ___   https://aws.amazon.com/linux/amazon-linux-2023/
   ~~       V/  /
     ~~   /
       ~~
[ec2-user@ip-10-20-1-10 ~]$ whoami && hostname -I
ec2-user
10.20.1.10
```

Technical Explanation: Validates the bastion jump model. The local workstation establishes an SSH tunnel through 13.233.42.18, and the bastion establishes an internal connection to 10.20.1.10 over the peering connection.

Test 5: Controlled Fault Injection & Recovery

Source Host	ewb-app-a (10.10.2.10)
Target Host	ewb-app-b (10.20.1.10)
Fault Action	Removed inbound ICMP Echo Request rule from ewb-sg-app-b
Recovery Action	Restored inbound ICMP rule for 10.10.2.10/32

```
[Fault Injected]: Inbound ICMP dropped at Security Group
$ ping -c 2 10.20.1.10 --> 100% packet loss (Routing intact, hypervisor firewall dropped packets)

[Fault Restored]: Inbound ICMP Echo Request re-authorized
$ ping -c 2 10.20.1.10 --> 0% packet loss, min/avg/max = 0.970/0.985/1.001 ms
```

Technical Explanation: Demonstrates security isolation depth. Routing delivered packets to App-B's elastic network interface (ENI), but the hypervisor-level security group firewall dropped them before reaching the OS kernel.

7. Technical Defense & Viva Answers

Q: Why is an explicit return route required on VPC B?
Defense: AWS route tables operate on a stateless, hop-by-hop basis. When App-A sends a request, VPC A's route table forwards it across the peering link. When App-B replies to 10.10.2.10, VPC B's route table independently determines how to deliver the return packet. Without a return route for 10.10.0.0/16 targeting the peering ID, VPC B's router has no destination target and drops the reply.

Q: Why is the SSH source IP in ewb-sg-app-b the bastion's private IP instead of the user's public IP?
Defense: Using SSH ProxyJump, the local machine opens a tunnel to the bastion's public IP. The bastion then originates a separate internal TCP connection to App-B across the peering link. Because this secondary connection originates from the bastion's network interface inside VPC A, the source IP observed by App-B is the bastion's private IP (10.10.1.10).

Q: If a VPC Peering Connection is in the 'Active' state, why might ping still fail?
Defense: The Active state confirms only that the peering handshake completed in the AWS control plane. Data plane transmission will still fail if route tables lack CIDR entries targeting the peering connection, security groups omit inbound ICMP permissions, or Network ACLs block the traffic.

Q: Can VPC B access the Internet through VPC A's Internet Gateway via Peering?
Defense: No. AWS VPC Peering does not support transitive routing (edge-to-edge routing). An Internet Gateway or NAT Gateway attached to one VPC cannot be utilized by a peered VPC. Centralized outbound egress requires an AWS Transit Gateway architecture.

8. Cleanup Evidence & Lessons Learned

All lab resources were deleted in reverse dependency order to prevent ongoing AWS billing:

- Terminated EC2 instances: ewb-bastion, ewb-app-a, and ewb-app-b (EBS root volumes automatically released).
- Removed VPC peering route entries (10.20.0.0/16 and 10.10.0.0/16) from all three custom route tables.
- Deleted VPC Peering Connection ewb-a-b-peer (pcx-0849ce714da9182ab).
- Deleted custom Security Groups: ewb-sg-bastion, ewb-sg-app-a, and ewb-sg-app-b.
- Detached and deleted Internet Gateway ewb-igw-a.
- Deleted subnets (ewb-a-public, ewb-a-private, ewb-b-private) and custom route tables, followed by VPCs ewb-vpc-a and ewb-vpc-b.

KEY LEARNINGS & ARCHITECTURAL TAKEAWAYS

Gained practical experience managing dual-VPC environments, configuring hop-by-hop asymmetric routing tables, and securing isolated instances with SSH ProxyJump without exposing private keys or private IPs to the internet. Understanding the clear separation between AWS control plane status (Active Peering) and data plane components (Route Tables, Security Groups, NACLs) is vital for enterprise cloud networking.